GDB 及其插件

GDB（GNU Debugger）是 GNU 提供的调试工具，可以在程序运行过程中查看寄存器、内存、栈、断点等状态。在 PWN 中经常使用 GDB 动态调试程序，观察程序执行流程和内存状态，辅助漏洞分析与 Exploit 编写。

基本使用

使用 GDB 打开程序：

```
gdb ./pwn
```

进入 GDB 后，可以使用：

```
run
```

或简写：

```
r
```

启动程序。

如果程序需要参数，也可以直接：

```
run arg1 arg2
```

断点

使用 `break` 或简写 `b` 设置断点。

在函数入口设置断点：

```
b main
b vuln
```

在指定地址设置断点：

```
b *0x401176
```

设置条件断点：

```
b *0x401176 if $rax == 0
```

查看当前断点：

```
info breakpoints
```

也可以简写：

```
i b
```

删除断点：

```
delete 1
```

禁用或启用断点：

```
disable 1
enable 1
```

程序执行

程序在断点处停止后，可以使用下面的命令控制执行：

命令	简写	作用
`continue`	`c`	继续运行直到下一个断点
`step`	`s`	单步执行，会进入函数
`next`	`n`	单步执行，不进入函数
`finish`	`fin`	运行到当前函数返回
`until`	`u`	运行到指定位置

在分析 PWN 程序时，最常用的一般是：

```
c
s
n
```

查看寄存器

查看所有寄存器：

```
info registers
```

也可以查看指定寄存器：

```
p $rax
p $rsp
p $rip
```

在 x86-64 程序中，经常关注：

```
RIP     当前执行的指令地址
RSP     当前栈顶
RBP     当前栈帧基址
RAX     常用于保存返回值
RDI     第一个函数参数
RSI     第二个函数参数
RDX     第三个函数参数
```

查看内存

GDB 使用 `x` 命令检查内存：

```
x/nfu ADDRESS
```

其中：

- `n`：查看多少个单位
- `f`：显示格式
- `u`：每个单位的大小

常见格式：

```
x     十六进制
d     十进制
s     字符串
i     指令
```

常见单位：

```
b     1 字节
h     2 字节
w     4 字节
g     8 字节
```

例如查看栈顶的 20 个 8 字节数据：

```
x/20gx $rsp
```

查看字符串：

```
x/s 0x404000
```

查看当前位置附近的汇编指令：

```
x/10i $rip
```

反汇编

使用：

```
disassemble main
```

或者简写：

```
disas main
```

查看函数的反汇编代码。

也可以指定地址范围：

```
disas 0x401100, 0x401200
```

查看调用栈

使用：

```
backtrace
```

或：

```
bt
```

查看当前函数调用栈。

例如：

```
#0  vuln()
#1  main()
```

可以帮助判断程序当前处于哪一层函数调用中。

观察点

除了普通断点，GDB 还支持观察某个变量或内存位置。

当值发生变化时停止：

```
watch expression
```

读取时停止：

```
rwatch expression
```

读取或修改时都停止：

```
awatch expression
```

在分析某个变量或内存地址什么时候被修改时比较有用。

Attach 到进程

除了直接使用 GDB 启动程序，也可以附加到已经运行的进程：

```
gdb -p PID
```

或者在 GDB 中：

```
attach PID
```

这在配合 Pwntools 调试时比较常用。

GDB 与 ASLR

使用 GDB 调试程序时，通常会关闭被调试程序的地址随机化，因此可能会发现每次运行时栈等地址保持不变。

可以通过：

```
show disable-randomization
```

查看相关设置。

如果希望调试时保留 ASLR，可以使用：

```
set disable-randomization off
```

常用命令速查

命令	作用
`r`	启动程序
`b`	设置断点
`c`	继续运行
`s`	单步进入
`n`	单步越过
`fin`	执行到当前函数返回
`i b`	查看断点
`info registers`	查看寄存器
`p`	打印表达式
`x`	查看内存
`disas`	反汇编
`bt`	查看调用栈

命令	作用
`watch`	设置观察点
`attach`	附加到进程

GDB 插件

原生 GDB 功能比较基础，在 PWN 中通常会配合插件使用。常见插件包括：

```
Pwndbg
GEF
PEDA
Pwngdb
```

其中 **Pwndbg** 是目前 PWN 中非常常用的 GDB 插件，可以自动显示寄存器、栈、反汇编、内存映射等信息。

###

Pwndbg

Pwndbg 是一个面向漏洞利用和逆向分析的 GDB 插件，在原生 GDB 的基础上增加了寄存器、栈、反汇编、内存映射、Heap 等信息的可视化显示，非常适合用于 PWN 调试。

安装：

```
git clone https://github.com/pwndbg/pwndbg
cd pwndbg
./setup.sh
```

安装完成后重新启动 GDB 即可。

context

程序在断点处停止时，Pwndbg 会自动显示当前程序的上下文信息，例如：

- 寄存器
- 当前汇编指令
- 栈
- 调用栈
- 函数参数

也可以手动输入：

```
context
```

或：

```
ctx
```

重新显示当前上下文。

还可以只查看某一部分：

```
context regs
context disasm
context stack
context backtrace
```

`context` 是 Pwndbg 中最核心的功能之一，程序每次停止时都会自动更新这些信息。Pwndbg

vmmap

使用：

```
vmmap
```

查看当前进程的虚拟内存映射。

通常可以看到：

```
程序代码段
Heap
libc
ld
Stack
```

以及每个内存区域对应的：

```
起始地址
结束地址
读写执行权限
文件映射
```

例如在分析 PIE、libc 地址或栈地址时，可以先通过 `vmmap` 确认对应内存区域。

Pwndbg 官方也推荐使用 `vmmap` 查看更加直观的进程内存映射。Pwndbg

telescope

`telescope` 用于连续查看并自动解析一段内存中的指针。

例如：

```
telescope $rsp
```

可以查看当前栈顶附近的内容。

也可以指定数量：

```
telescope $rsp 20
```

与原生 GDB 的：

```
x/20gx $rsp
```

相比，`telescope` 会自动继续解析可能的指针，因此在查看栈和指针链时更加直观。Pwndbg

checksec

使用：

```
checksec
```

可以查看当前二进制程序启用的安全保护，例如：

```
RELRO
Stack Canary
NX
PIE
```

在开始分析 PWN 题目时通常可以先执行一次 `checksec`，了解程序启用了哪些保护机制。

search

使用：

```
search
```

可以在进程内存中搜索字符串、整数、字节序列或指针。

例如搜索字符串：

```
search "/bin/sh"
```

常用于在 libc 或其他内存区域中寻找需要的数据。Pwndbg 的 `search` 支持搜索字符串、字节序列、指针以及整数值。Pwndbg

canary

如果程序开启了 Stack Canary，可以使用：

```
canary
```

辅助查看当前进程中的 Canary 值。

在分析栈溢出程序时，可以配合：

```
checksec
canary
```

快速确认 Canary 保护以及对应的值。Pwndbg

Heap 相关命令

Pwndbg 对 glibc Heap 提供了大量辅助功能。

查看 Heap Chunk：

```
heap
```

`heap` 会从当前线程的 Heap 中依次解析 Chunk，可以用于观察 Chunk 的大小、状态和布局。Pwndbg

查看各类 Bin：

```
bins
```

`bins` 会显示当前 Arena 中的 Bin 以及当前线程的 tcache 信息。Pwndbg

在 Heap PWN 中，经常会使用类似：

```
heap
bins
tcachebins
fastbins
unsortedbin
```

观察 Chunk 在不同 Bin 中的状态。

常用 Pwndbg 命令

命令	功能
`context` / `ctx`	查看当前调试上下文
`vmmap`	查看进程内存映射
`telescope`	查看并解析连续内存和指针
`checksec`	查看程序安全保护
`search`	在内存中搜索数据
`canary`	查看 Stack Canary
`heap`	查看 Heap Chunk
`bins`	查看 Heap Bin 和 tcache
`procinfo`	查看进程相关信息
`xinfo`	查看某个地址对应的内存信息

对于刚开始使用 Pwndbg 的同学，优先熟悉：

```
context
vmmap
telescope
checksec
search
heap
bins
```

```
D1NO{pwndbg_m4k3s_gdb_m0r3_pwn4bl3}
```